

Question-4(c)

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the range of a values
a_values = np.linspace(-5, 5, 100)

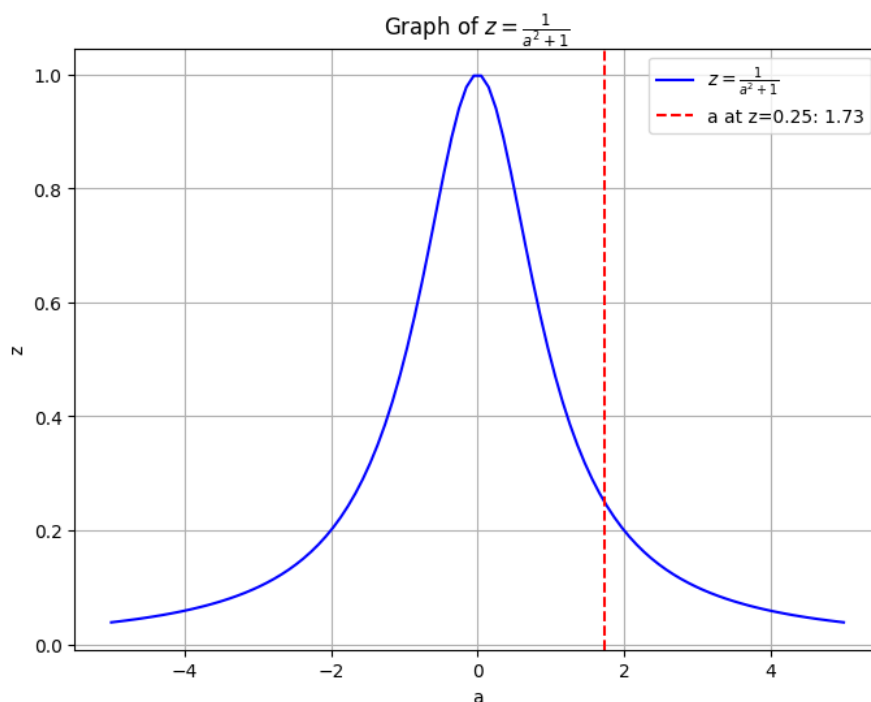
# Compute z values
z_values = 1 / (a_values**2 + 1)

# Plot the graph
plt.figure(figsize=(8, 6))
plt.plot(a_values, z_values, label='$z = \frac{1}{a^2 + 1}$', color='blue')
plt.xlabel('a')
plt.ylabel('z')
plt.title('Graph of $z = \frac{1}{a^2 + 1}$')
plt.grid(True)
plt.legend()

# Find value of a when z = 0.25
a_at_z_025 = np.sqrt(1 / 0.25 - 1)
plt.axvline(a_at_z_025, color='red', linestyle='--', label=f'a at z=0.25: {a_at_z_025:.2f}')
plt.legend()

plt.show()
```

Graph:



Question-6

Code:

```
from sympy import symbols, integrate

x, y, z = symbols('x y z')

# Function definition
expr = x**2 + y**2 + z**2

# x limits
a = 0
b = (12 - 3*y - 4*z) / 2

# y limits
c = 0
d = (12 - 4*z) / 3

# z limits
e = 0
f = (12) / 4

# Integration
in_1 = integrate(expr, (x, a, b)) # integrate wrt x
in_2 = integrate(in_1, (y, c, d)) # integrate wrt y
in_3 = integrate(in_2, (z, e, f)) # integrate wrt z

# Simplifying result
result = in_3.evalf()

# Printing
print("Result =", result)
```

Result:

73.2000

Question-8

Code:

```
#----- Importing libraries -----
import numpy as np
import matplotlib.pyplot as plt
import math
from mpl_toolkits.mplot3d import Axes3D

#----- Defining function -----
def E(x, y, z):
    return pow(x,2) + pow(y,2) # Use numpy functions for element-wise operations

#----- x & y values -----
x = np.linspace(-3, 3, 50)
y = np.linspace(-3, 3, 50)
X, Y = np.meshgrid(x, y) # Generate meshgrid for X and Y

Z = E(X, Y, 0) # Calculate Z values using the function E

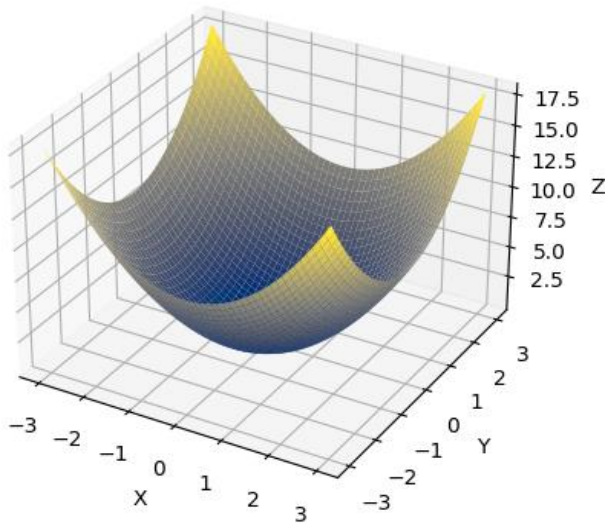
#----- Plotting surface -----
figure = plt.figure()
graph = figure.add_subplot(111, projection='3d')
graph.plot_surface(X, Y, Z, cmap='cividis')

#----- Labelling -----
graph.set_xlabel('X')
graph.set_ylabel('Y')
graph.set_zlabel('Z')
graph.set_title('Question - 8')

plt.show()
```

Graph:

Question - 8



Question-9

Code:

```
#----- Importing libraries -----  
import numpy as np  
import matplotlib.pyplot as plt  
import math  
from mpl_toolkits.mplot3d import Axes3D  
  
#----- Defining function -----  
def E(x, y, z):  
    return pow(x,2) + pow(y,2) # Use numpy functions for element-wise operations  
  
#----- x & y values -----  
x = np.linspace(-3, 3, 50)  
y = np.linspace(-3, 3, 50)  
X, Y = np.meshgrid(x, y) # Generate meshgrid for X and Y  
  
Z = E(X, Y, 0) # Calculate Z values using the function E  
  
#----- Plotting surface -----
```

```

figure = plt.figure()
graph = figure.add_subplot(111, projection='3d')
graph.plot_surface(X, Y, Z, cmap='cividis')

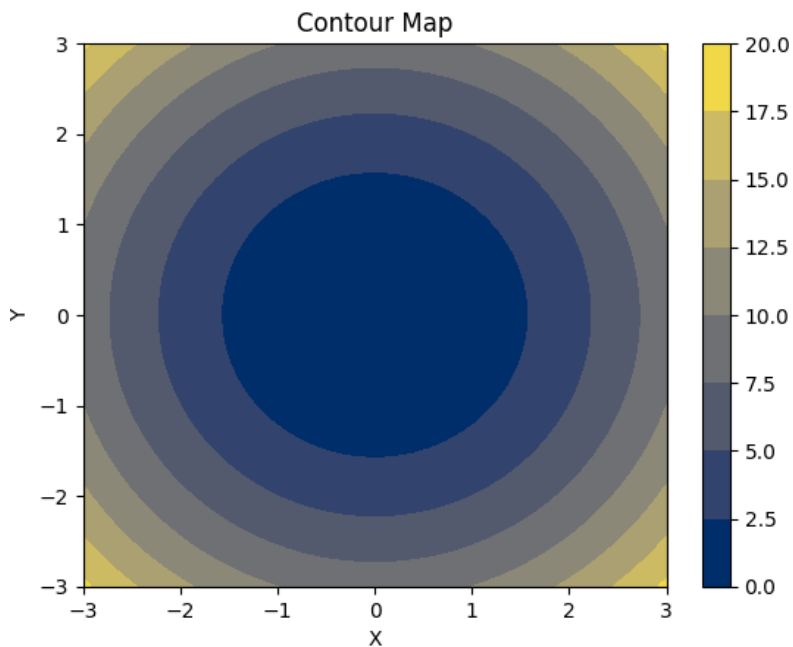
#----- Plotting contour map -----
plt.figure()
contour_map = plt.contourf(X, Y, Z, cmap='cividis')
plt.colorbar(contour_map)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Contour Map')

#----- Labelling -----
graph.set_xlabel('X')
graph.set_ylabel('Y')
graph.set_zlabel('Z')
graph.set_title('Question - 9')

plt.show()

```

Graph:



Question-10

Code:

```
#----- Importing libraries -----
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

#----- Defining function -----
def E(x, y):
    return x**2 + y**2

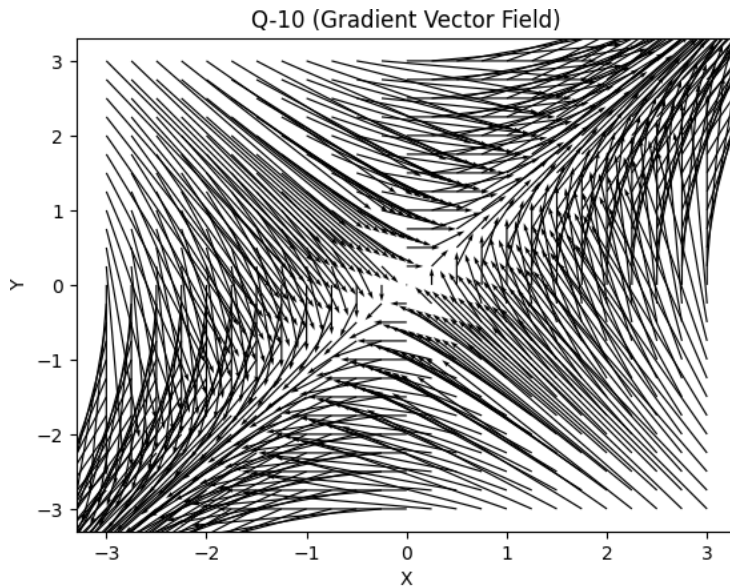
#----- x & y values -----
x = np.linspace(-3, 3, 25)
y = np.linspace(-3, 3, 25)
X, Y = np.meshgrid(x, y) # Generate meshgrid for X and Y

#----- Calculate gradient -----
dx, dy = np.gradient(E(X, Y), x, y)

#----- Plotting gradient vector field -----
plt.figure()
plt.quiver(X, Y, dx, dy, scale=20)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Q-10 (Gradient Vector Field)')

plt.show()
```

Graph:



Question-11

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Objective function
def f(x1, x2, x3):
    return (x1 - 4)**4 + (x2 - 3)**2 + 4*(x3 + 5)**4

# Gradient of the objective function
def gradient_f(x1, x2, x3):
    return np.array([4*(x1 - 4)**3, 2*(x2 - 3), 16*(x3 + 5)**3])

# Steepest descent method
def steepest_descent(x, alpha, iterations):
    path = [x]
    for i in range(iterations):
        x = x - alpha * gradient_f(*x)
        path.append(x)
    return np.array(path)
```

```

# Initial point and parameters
initial_point = np.array([4, 2, -1])
alpha = 0.1
iterations = 3

# Perform steepest descent method
path = steepest_descent(initial_point, alpha, iterations)

# Plotting the objective function
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Generate grid points
x1_vals = np.linspace(0, 8, 100)
x2_vals = np.linspace(0, 6, 100)
x1_vals, x2_vals = np.meshgrid(x1_vals, x2_vals)
x3_vals = np.linspace(-6, 0, 100)
z_vals = f(x1_vals, x2_vals, x3_vals)

# Plot objective function surface
ax.plot_surface(x1_vals, x2_vals, z_vals, cmap='viridis', alpha=0.7)

# Plot path taken by steepest descent method
path = np.array(path)
ax.plot(path[:, 0], path[:, 1], path[:, 2], marker='o', color='red', label='Steepest
Descent Path')

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('f(x1, x2, x3)')
ax.set_title('Objective Function and Steepest Descent Path')
ax.legend()
plt.show()

```


Graph:

Objective Function and Steepest Descent Path

